# Appeasing the Machine Spirit: RLHF in In-Context Learning

03 Nov 2026

*Two rites for predicting, and then preventing, a solo developer's playtest notes.*

I'm making a game alone: *The Good Vet*, an open-world vet game with soulslike fights. Every few days I play the latest build and write down everything that bothers me. Then the list goes back into the todo, the todo gets built, and I play again.

After a month there were sixteen of those lists in git, each one written against a todo. That's a dataset. So I asked an agent two questions. Can you read the todo and predict what I'll write? And if you can, can you change the todo so I don't have to write it?

## Two rites

Nothing is fine-tuned. The "weights" are one markdown file in the agent's memory folder. The "dataset" is git tags pairing each todo with its list. Each epoch is a fresh agent, cleansed of all prior context. That's the `zero_grad()`, so the file is the only thing that carries over.

**The First Rite** trains the memory. An agent reads the memory, forecasts each past list from its todo, opens the real list, writes down what it got wrong, and rewrites the memory.

**The Second Rite** freezes the memory and changes the todo instead. It forecasts what I'll complain about once the todo is built, writes those complaints into the todo as fixes, and forecasts again, until nothing new comes up. It's like optimising the input against a frozen network.

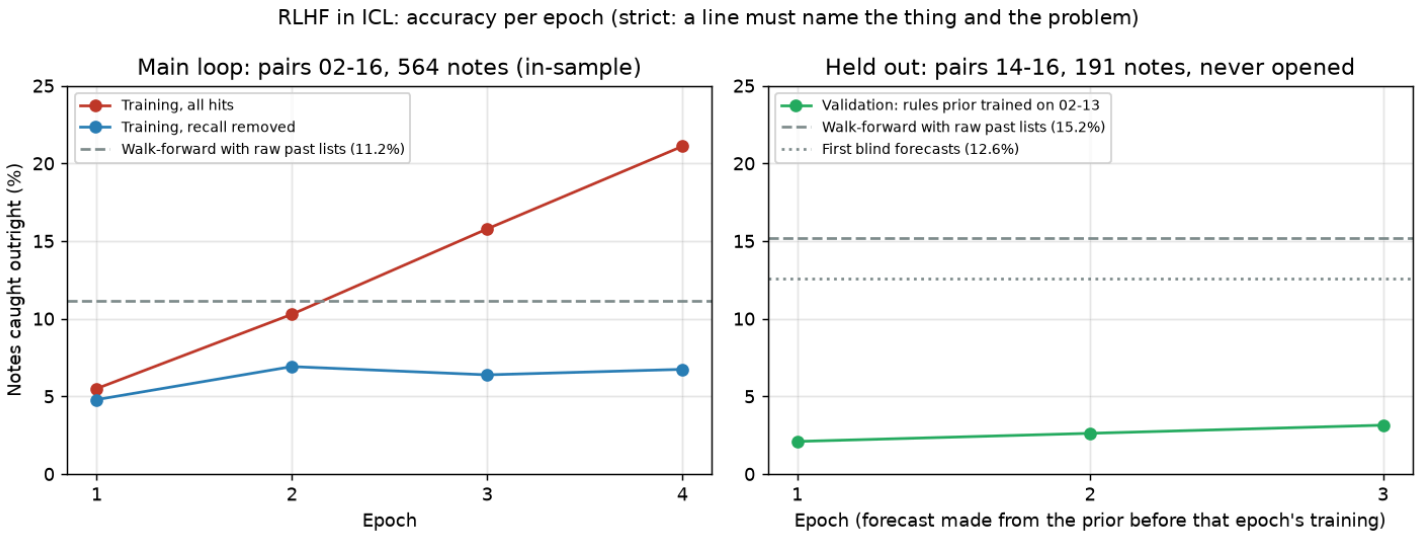
If both rites work, my next playtest list comes back empty, and the machine spirit is appeased.

```
The First Rite: train the memory on past (todo, list) pairs
  for each epoch:
    agent = fresh agent                       # zero_grad()
    for each past pair, in order:
      forecast = agent.predict(memory, todo, earlier lists)   # before the list is opened
      loss     = agent.compare(forecast, real list)          # a count, and what went wrong
    memory = agent.rewrite(memory, patterns in the losses)  # do/don't rules, no past notes
    commit memory                             # checkpoint
    stop when held-out hits stop rising

The Second Rite: freeze the memory, converge the todo
  repeat:
    agent    = fresh agent
```

```
    forecast = agent.predict(memory, todo, every past list) # as if the todo were built
    new      = lines the todo doesn't cover yet             # the loss
    todo     = fold(todo, new)              # boxes, watch lines, decisions
    commit todo
  until new is (almost) empty
```

## The curve that lied




*Left: training rose, but with recall removed it was flat from epoch 2. Right: on three lists the memory never saw, a rules memory caught about 3%, while an agent reading the raw earlier lists caught 15%.*

Training accuracy went from 5.5% of my notes to 21% in four epochs, which looked great. Then the scorer pointed out that many hits repeated my notes almost word for word. Each epoch, the agent had written my past notes into the memory: offender lists, per-tool wish lists. By epoch four, about 81 of its 119 hits were recall. With that removed, the curve was flat from epoch two.

> "This is cheating. You can't just write out the diff in the memory itself."

So we held out my last three lists and trained a fresh memory on the rest. On the lists it had never seen, it caught 4, 5 and 6 notes out of 191: about 3%. An agent with no rules memory at all, reading my earlier lists raw, caught 29. Five times better.

## What actually predicts me

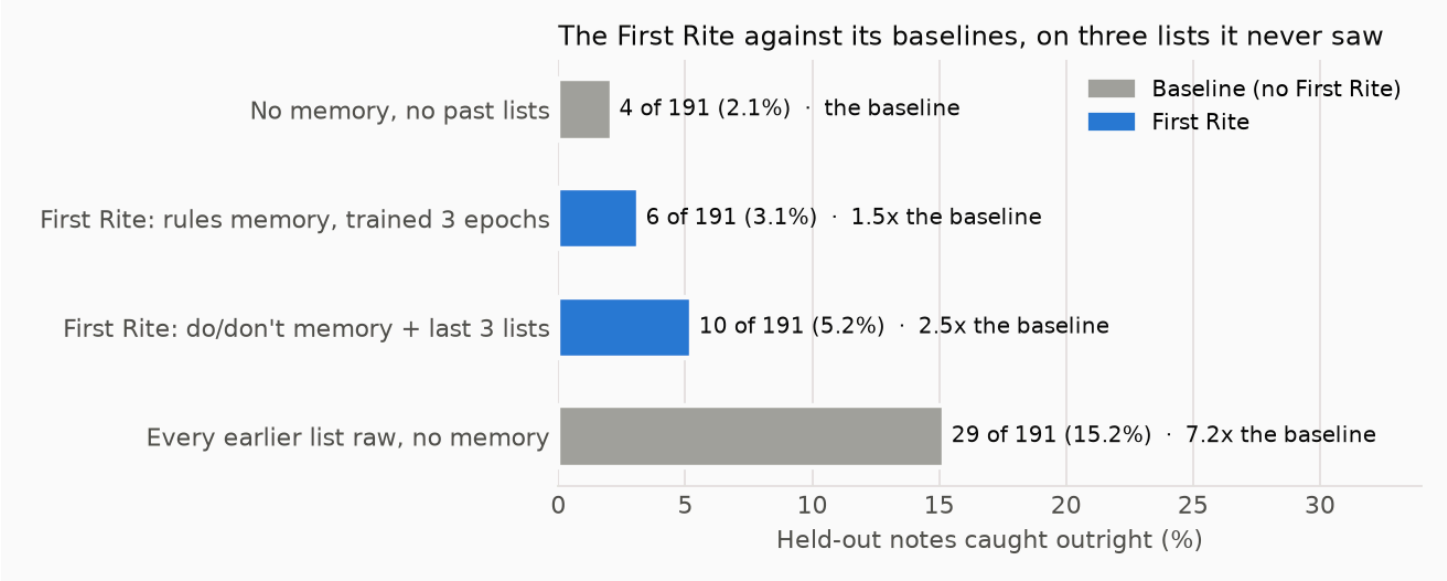
Most of my notes are repeats of named things, half-finished asks, or the next step of a wish I already made. Those live in the raw history, not in rules about the history. A rule like "fixes checked only by a test come back" is true, but it doesn't say *which* thing will come back.

So the memory was rewritten as general do/don't pairs, each rule with its opposite, with none of my past notes in it. Frozen and tested, it caught 5.7% with zero recall, and twice the held-out score of the trained rules. Most of its misses were rules it already had but no room to write, because I'd capped it at twelve lines. A predicted problem I don't mention costs nothing. It could be an oversight on my part.

| Run | Training (564 notes) | Held out (191) |
|---|---|---|
| First Rite, epoch 4, all hits | 119 | n/a |
| First Rite, epoch 4, recall removed | ~38 | n/a |
| Validation, rules trained on the rest | n/a | 4 to 6 |
| Raw earlier lists, no rules memory | 63 | 29 |
| Do/don't memory, frozen | 32 | 10 |

## Against the baselines

The fair test is the three lists the memory never saw (191 notes). Two baselines don't use the First Rite at all. One gets no memory and no past lists. The other gets every earlier list raw and no memory.




*Grey bars are baselines without the First Rite; blue bars use it. Percentages are held-out notes caught outright.*

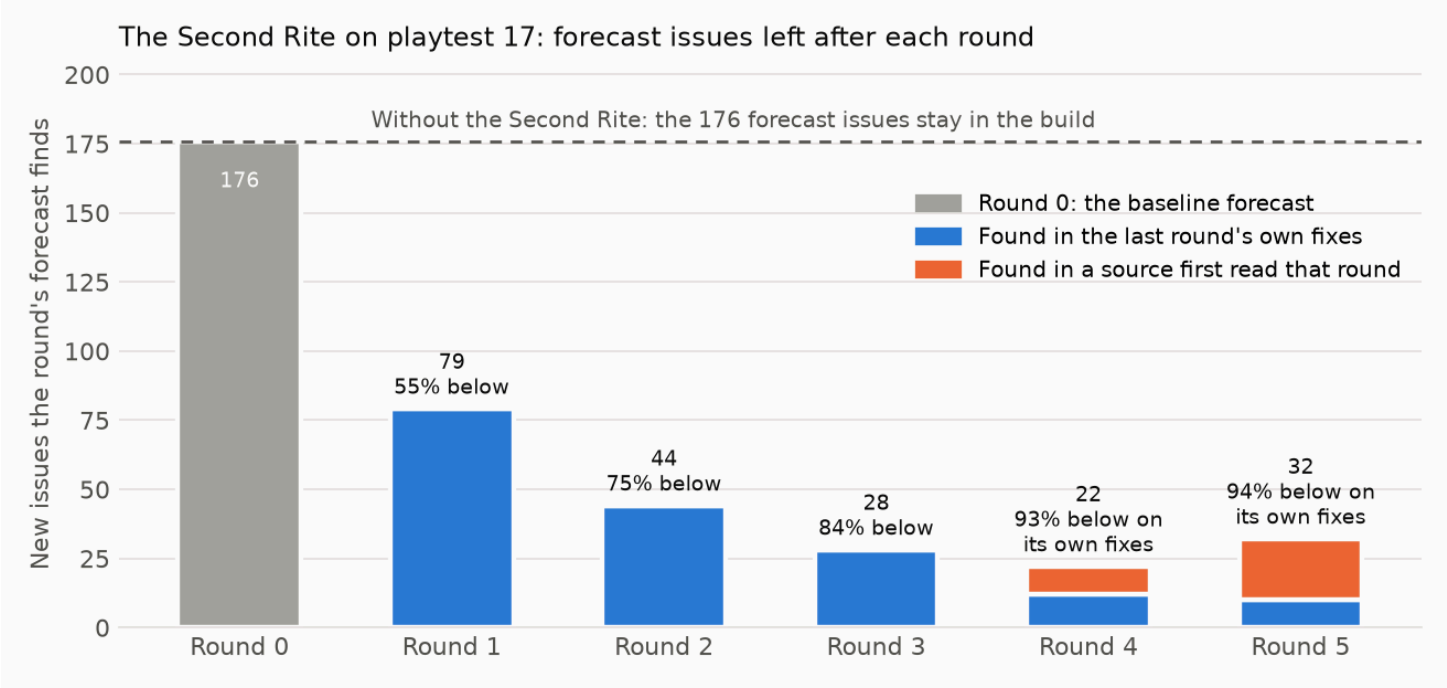
| Held out, 191 notes | Caught | vs the no-memory baseline |
|---|---|---|
| Baseline: no memory, no past lists | 4 (2.1%) | 1.0x |
| First Rite: rules memory, trained 3 epochs | 6 (3.1%) | 1.5x (+50%) |
| First Rite: do/don't memory + last 3 lists | 10 (5.2%) | 2.5x (+150%) |
| Baseline: every earlier list raw, no memory | 29 (15.2%) | 7.2x (+625%) |

So the First Rite beats having nothing, by 50% with trained rules and by 150% with the do/don't memory. But it doesn't beat simply reading my old lists, which is 2.9 times better than the do/don't memory on its own. That's why the memory now says to read every earlier list raw, and spends its rules on how to read them. That combination (the do/don't memory plus every earlier list, with no cap on lines) hasn't been scored yet. My next playtest scores it.

# Turning the forecast into work

The Second Rite took the frozen memory, today's todo and all sixteen lists, and forecast 176 notes for my next playtest. Twenty-two turned out to be already handled in the code. The rest became 64 fix boxes, 35 warnings on existing boxes, and 9 decisions with defaults. Every box says how it gets checked without me playing.

Then the rite ran again on the new todo, with a fresh agent each round, for five rounds. The baseline is not running it: the todo stays as it was, and the forecast's 176 issues stay in the build.




*Blue is what each round found in the last round's fixes. Orange is what it found in a source it read for the first time (my chapter docs). The dashed line is the baseline: no Second Rite.*

| Round | New issues found | From the last round's fixes | vs the 176 baseline |
|---|---|---|---|
| 0 (the baseline forecast) | 176 | n/a | n/a |
| 1 | 79 | 79 | 55% fewer |
| 2 | 44 | 44 | 75% fewer |
| 3 | 28 | 28 | 84% fewer |
| 4 | 22 | 12 | 88% fewer (93% on its own fixes) |
| 5 | 32 | 10 | 82% fewer (94% on its own fixes) |

Most of each round's finds were faults in the last round's fixes, which is what you'd expect: every fix adds a little surface. On that surface alone it converged, from 79 to 10. The bump at the end came from the agent finally reading my chapter docs, which list the parts I hadn't tested. A new source behaves like new data. Next time, round zero gets every source at once.

One caveat matters. These are issues a forecaster expects me to find, not issues I found. "94% fewer" means the frozen memory can't think of much more to complain about. Whether I can is the next playtest. What's left is what only playing can find: how it feels, the prices, the numbers, and a real friend on a real connection.

The real test is my next playtest. If the list is short, the spirit is appeased. If it isn't, the list becomes pair seventeen, and the First Rite begins again.

## What I'd tell you

Measure recall, or your curve will lie. Keep the labels out of the weights. Give the model the raw history, and spend the memory on how to read it. And a fresh agent per epoch is expensive, because onboarding is costly, just like real life.

---

*The method, the numbers and the prompts are in the preprint. The LaTeX source, the original page and the CI that builds both are on GitHub. The experiments were run by Claude Opus 5.5 agents in Claude Code.*

---


**What do you think?**

0 Responses

Upvote Funny Love Surprised Angry Sad

**0 Comments** Login ▾

Start the discussion...

LOG IN WITH OR SIGN UP WITH DISQUS

Name

♡ • Share Best Newest Oldest

**All articles**


Appeasing the Machine Spirit: RLHF in In-Context Learning
03 Nov 2026

Four Tricks at the Edge of Formal Language Theory
15 Aug 2026

Spring-Loaded Dinosaurs: What Penguins, Kiwis, and Armored Core Get Right (and Wrong) About Legs
28 Feb 2026

Passive Velocity Sorting: A Geometric Maxwell's Demon?
14 Dec 2025

AC Gravity and Gravitational Opacity
05 Dec 2025

Theorem: Equivalence of -arccoth and tan Functions
21 Nov 2025

The Photonic Transformer Hypothesis: Rethinking Photosynthesis and Methane Consumption in Plants
19 Nov 2025

The Scalable Quasi-Perpetual Photonic Machine
17 Nov 2025

How to Make a Ball Orbit Itself
08 Nov 2025

The Boron-Nitrogen Catastrophe: A Critical Gap in Beta Decay Verification
28 Oct 2025

X-Ray Data Pipeline: Ultra-High-Speed Communication Through Limestone Tubes
26 Oct 2025

The Universal Approximation Theorem Is Right. You're Using It Wrong.
19 Oct 2025

The Power Law Illusion: A Measurement Artifact Hypothesis
11 Oct 2025

Fractional Gamma Function via Fractional Derivatives
18 Sep 2025

On the cruel irony of the P-NP problem
18 Dec 2023

The Journey from India to Germany: A Guide for IT Professionals
20 Jun 2023

How does AutoGPT work under the hood?
03 May 2023

Unit Test Recorder - Automatically generate unit tests as you use your application
25 Jun 2020

How to migrate from vanilla Kubernetes to Istio service mesh?
14 Oct 2019